Report

1.0

Customer

Exactly



Branch review

Flash Loan Adapter and Inverse Price Feed

October 19, 2026

ABDK Consulting

Contents

1. Document Revisions 3

2. Introduction 3

3. System Overview 4

4. Methodology 6

5. Findings 7

5.1. Minor 8

6. Conclusion 25

About Us 27

Contacts 27

1. Document Revisions

#	Date	Author	Description
0.1	October 9, 2026	Dmitry Khovratovich	Initial report draft.
1.0	October 19, 2026	Dmitry Khovratovich	Public report. Client responses incorporated; fixes reviewed at commit `9bf6024`.

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

ABDK Consulting was engaged by Exactly to review two changes on branch `morpho-flashloan-adapter` of exactly/protocol: a new price feed, `PriceFeedInverse`, that inverts a Chainlink answer, and a rewrite of the flash loan adapter on top of Morpho Blue. The review covered commit `371e41a`, measured against `0d1ca2a`, the merge base with `main`. The client declared `contracts/PriceFeedInverse.sol` and `contracts/periphery/FlashLoanAdapter.sol` as the scope.

Other files were read only as context and are not part of the audited scope. These include `Auditor.sol`, `DebtRoller.sol`, `IPriceFeed.sol`, the other price feed wrappers, the previous Balancer V3 adapter, Morpho Blue and the ExaPlugin contracts that take flash loans. Reading them was necessary to confirm how the two contracts are consumed, but no assurance is offered on their contents.

A draft of this report was delivered to the client, who responded to every finding. The client's response is reproduced verbatim beneath each finding. Where a fix was claimed, the referenced commit was examined, and the fix round as a whole was reviewed at

commit `9bf6024`, including the new code it adds. This review is a static reading of the source. No code was compiled, executed, or tested as part of it.

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

The Exactly protocol is a lending protocol whose `Auditor` contract values account positions through Chainlink-style price feeds exposed by the `IPriceFeed` interface, and whose periphery contracts let users manage debt within a single transaction. This review covers two such pieces: a price feed adapter that inverts the quote direction of an existing feed, and a flash loan adapter that gives `DebtRoller` and the ExaPlugin (exactly/exa) a uniform way to borrow tokens for the duration of one transaction.

Under review is the repository exactly/protocol at commit `371e41a5e2f3429301ecd7dd547bd36da38385de` of branch `morpho-flashloan-adapter`, taken against base commit `0d1ca2a6b753f40c4a0c5a442ee25246122594f7`, the merge base with `main`. The branch has two commits: `2e0b944` adds `PriceFeedInverse` and `371e41a` switches the adapter to Morpho. The scope is `contracts/PriceFeedInverse.sol` and `contracts/periphery/FlashLoanAdapter.sol`, 53 nSLOC in total (19 and 34), and the adapter is a full rewrite of the previous Balancer V3 version. According to the client, `PriceFeedInverse` is needed to list wARS as a new non-collateral market on Base, which requires an ARS/USD price while Chainlink offers only a USD/ARS feed there, quoted in ARS per USD with 8 decimals. The new adapter replaces Balancer V3, which is winding down, and keeps the same `flashLoan` interface. Its main use is to let the ExaPlugin take USDC flash loans for repay and cross repay, and for debt rollover through `DebtRoller`.

`PriceFeedInverse` implements `IPriceFeed`, which declares only `decimals` and `latestAnswer`. Its immutables are the source feed `priceFeed`, `decimals`, copied from the source feed in the constructor, and `baseUnit`, ten to the power `decimals`. The function `latestAnswer` converts the source answer to `uint256` with Solady `SafeCastLib`, which reverts on a negative value, and returns `baseUnit` squared divided by that answer, rounded up with `mulDivUp`; a zero answer reverts in the division. The contract has no admin and no round data. `Auditor` consumes the feed through `assetPrice`, which calls `latestAnswer` and scales the result to 18 decimals, and requires `decimals` to equal `priceDecimals`, which is 8 on Base, whenever a feed is set.

`FlashLoanAdapter` implements `IFlashLoaner`, with the interfaces imported from `DebtRoller.sol`. It is stateless: its only variable is the immutable `morpho`, a Morpho Blue instance, and it has no admin. The function `flashLoan` is permissionless; it requires exactly one token and one amount and calls `morpho.flashLoan` with a payload encoding the recipient, tokens, amounts and data. Morpho then calls `onMorphoFlashLoan`, which accepts calls only from `morpho`, decodes the payload, transfers the loan to the recipient, calls the recipient's `receiveFlashLoan` with a fee of zero because Morpho flash loans are free, and finally approves `morpho` for the loan amount, which Morpho pulls back. A new view, `available`, returns Morpho's balance of a token. The ExaPlugin and `DebtRoller` each authenticate the callback with `msg.sender` and a single-use stored request hash. Relative to the Balancer V3 version, the timelocked `AccessControl` admin, the ERC-4626 wrapped-token fallback with `setWToken`, the `InsufficientLiquidity` error and Balancer's push-style settlement are removed. Morpho Blue is deployed on Base and OP Mainnet, and the deploy script now targets any network with a Morpho deployment, so OP Mainnet becomes eligible.

The client's fixes were reviewed at commit `9bf6024efb02cc556309a14d2e15e03f7beedc08` of the same branch, six commits ahead of the reviewed one, which change only the two in-scope files, their tests and the gas snapshot. In `PriceFeedInverse`, `baseUnit` is replaced by `baseUnitSquared`, the square of the base unit computed once in the constructor, so `latestAnswer` performs a single rounded-up division and a source with more than 38 decimals is rejected at deployment. The source is now typed as an `IAggregatorProxy` interface, and the feed forwards its round data: `latestRoundData` and `getRoundData` return the source's round id, timestamps and `answeredInRound` unchanged, with the answer inverted as in `latestAnswer`, except that `getRoundData` keeps a zero answer, which the source returns for rounds without data, as zero; `phaseId` returns the source's phase. The `Auditor` still reads only `latestAnswer`. In `FlashLoanAdapter`, `flashLoan` takes `calldata` parameters, and the Morpho payload carries only the recipient, the token and the data; `onMorphoFlashLoan` takes the loan amount from Morpho's `assets` argument and rebuilds the one-element arrays it passes to `receiveFlashLoan`. The `flashLoan` selector and the arguments that recipients receive are unchanged.

/ (root directory)
PriceFeedInverse.sol

/periphery/
FlashLoanAdapter.sol

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the Flash Loan Adapter and Inverse Price Feed project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment
The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis
Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis
For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis
The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical
Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major
Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate
Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor
Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the Flash Loan Adapter and Inverse Price Feed codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Minor	**16**	**4**

Fixed 4 out of 16 issues reported.

5.1. Minor

CVF-1: acknowledged Category: **procedural**

Any token balance held by the adapter can be taken by anyone, through a loan that this balance repays.

`onMorphoFlashLoan` sends `amounts[0]` to the recipient, calls its `receiveFlashLoan`, and then approves Morpho to pull `amounts[0]` back from the adapter. It never checks that the recipient has actually returned the tokens. Morpho's final `safeTransferFrom` only requires the adapter to hold enough tokens, whatever their origin. Any balance that was already in the adapter therefore repays a loan on the recipient's behalf.

`flashLoan` is permissionless and accepts any recipient, so anyone can turn such a balance into a free loan that is never repaid:

1. Tokens end up in the adapter: a mistaken direct transfer, an airdrop, or a caller that returns more than it borrowed. `DebtRoller` and the ExaPlugin currently return exactly `amounts[0] + fees[0]`, but nothing in the adapter enforces this.
2. The attacker calls `flashLoan(attacker, [token], [balance], "")`, where `attacker` is a contract whose `receiveFlashLoan` does nothing.
3. Morpho sends `balance` to the adapter, the adapter forwards it to `attacker`, and the callback returns without repaying.
4. The adapter approves Morpho, and Morpho pulls `balance` out of the stray tokens. The attacker keeps the loaned amount.

The adapter has no administrator and no sweep function, so the protocol cannot recover such balances; they go to whoever takes them first. Nested loans cannot extract more than the stray balance, because every frame's final pull still needs its full amount. The Balancer V3 adapter had the same property.

Recommendation

Verify the repayment explicitly by comparing the adapter's own balance before forwarding the loan and after the callback, for example:

```
uint256 expected = token.balanceOf(address(this)); // already
  includes the loan
token.safeTransfer(recipient, amounts[0]);
IFlashLoanRecipient(recipient).receiveFlashLoan(tokens, amounts,
  new uint256[](1), data);
uint256 balance = token.balanceOf(address(this));
```

```
if (balance < expected) revert NotRepaid(token, expected,
  balance);
token.safeApprove(address(morpho), amounts[0]);
```

The check also holds for nested loans of the same token, because each frame compares against its own snapshot. In addition, or instead, add a permissionless sweep that sends any surplus to a fixed treasury address set at construction.

Client Comment

Acknowledged. The adapter is not meant to hold tokens, and both callers return exactly amounts[0] + fees[0]. A sweep would add an admin or treasury setting to a contract designed to have none, and the repayment check would add two balanceOf calls to every loan. This keeps the same property as the previous adapter.

/periphery/FlashLoanAdapter.sol

```
43      token.safeTransfer(recipient, amounts[0]);
44      IFlashLoanRecipient(recipient).receiveFlashLoan(tokens, amounts,
    new uint256[](1), data);
45
46      token.safeApprove(address(morpho), amounts[0]);
```

CVF-2: acknowledged Category: **behavior**

A loan larger than Morpho's balance of the token reverts with Morpho's generic `transfer reverted` message, and Morpho on Base holds no wARS at all.

`flashLoan` passes every request straight to `morpho.flashLoan`. When Morpho holds less than `amounts[0]`, the token transfer inside Morpho fails and Morpho reverts with the string `transfer reverted`; the repository's own test `test_flashLoan_reverts_whenOverAvailable` expects exactly that. The Balancer V3 adapter first compared the request with the vault's balance and could fall back to an ERC-4626 wrapped token. Only if neither had enough did it revert, with `InsufficientLiquidity()`. The new adapter exposes `available(token)`, but neither `flashLoan` nor its callers use it.

This matters most for the market this release is meant to enable. Morpho Blue on Base held no wARS at block 52552816, and no Morpho market lists it. The ExaPlugin's repay and cross repay, and `DebtRoller` rolls, all borrow the debt market's own asset, so every flash-loan-based repay or roll of wARS debt reverts. Balancer V3 holds no wARS either, so this is a limitation of the wARS market rather than a regression. Nothing in the adapter or its callers reports it, though, and accounts with wARS debt have to repay directly from their own funds.

On OP Mainnet, where `deploy/FlashLoan.ts` now also deploys the adapter, the switch would raise rather than lower capacity for the plugin's native USDC loans: at block 158148107 Morpho held about 72,124 USDC, against about 5,702 USDC in the Balancer V2 vault that the live plugin uses. Morpho holds no OP or USDC.e, but the live

plugin does not borrow those. The Exa app's repay screen on OP caps the amount by the Balancer V2 vault's balance, so that cap would have to follow the switch.

Recommendation

Check liquidity in `flashLoan` and revert with a descriptive error:

```
uint256 liquidity = tokens[0].balanceOf(address(morpho));
if (amounts[0] > liquidity) revert
  InsufficientLiquidity(tokens[0], amounts[0], liquidity);
```

Before listing wARS, decide how wARS borrowers are meant to repay and roll: either arrange wARS liquidity on Morpho, or document that flash-loan-based repay, cross repay and roll are unavailable for wARS and have the app offer direct repayment instead. Before switching the OP Mainnet plugin, base the app's repay cap on `available(token)` rather than on the Balancer V2 vault's balance.

Client Comment

Acknowledged. The check would only change the revert message, and it would cost an extra balanceOf call on every loan. Off-chain components will use available() to cap flash-loan amounts, and wARS debt will be repaid directly instead of through flash loans.

/periphery/FlashLoanAdapter.sol

```
30     morpho.flashLoan(address(tokens[0]), amounts[0],
abi.encode(recipient, tokens, amounts, data));
```

CVF-3: acknowledged Category: **behavior**

The inverse feed cannot pass the source's update time through, and nothing bounds the age of the price it returns.

`latestAnswer` reads `priceFeed.latestAnswer()` and inverts it, so it never sees the source's `updatedAt`. Because `PriceFeedInverse` implements only `IPriceFeed`, it cannot expose `latestRoundData` to anyone else either. `Auditor.assetPrice` applies no staleness check of its own.

The source is Chainlink's USD/ARS feed on Base, read through this proxy:

```
0x9eb8a54d0590798880C665C7A6d51B95f4078Ad7
```

According to Chainlink's documentation, it is a 24/7 feed built from a single provider's crypto-derived exchange rate, which can diverge from the official rate. It updates on a 0.5% deviation threshold or a 24-hour heartbeat. Over the 132 rounds published between 23 July and 12 October 2026, the longest gap between rounds was 24.0 hours, weekends included. In normal operation the answer is therefore at most about a day old, but the peso can move a lot within a day: against the crypto dollar it appreciated by about 20% on 23 October 2023. If the single provider stalls or the feed is deprecated, the last answer stays in place indefinitely, and nothing on-chain notices.

For a debt-only market, the risk depends on the direction of the move:

- If the peso depreciates while the feed is stale, wARS is overpriced, so debt is overvalued. That is safe for solvency, but it brings liquidations forward and makes `calculateSeize` seize more collateral than the true price warrants.
- If the peso appreciates while the feed is stale, wARS is underpriced. Accounts can borrow more wARS than their collateral supports until the next update, and bad debt follows if the jump exceeds the margin left by the adjust factors.

Recommendation

Implement `latestRoundData()` in `PriceFeedInverse`, returning the inverted answer together with the source's `roundId`, `startedAt`, `updatedAt` and `answeredInRound`. Off-chain monitoring, and any future staleness-aware consumer, can then see when the price was last updated.

Consider an immutable maximum age, slightly above the 24-hour heartbeat, that makes `latestAnswer` revert when the source is older. Such a revert also blocks borrows, collateral withdrawals and liquidations for every account in the wARS market, so pair it with monitoring and a procedure for freezing wARS borrowing. Chainlink itself recommends freshness checks, value bounds and monitoring for this feed.

Client Comment

Partially fixed in 9bf6024efb02cc556309a14d2e15e03f7beedc08. PriceFeedInverse now implements latestRoundData, getRoundData and phaseId. The rest is acknowledged. The protocol reads prices through latestAnswer and does not check how recent they are. This is an accepted trade-off. PriceFeedInverse follows the same design as the protocol's other price feeds.

/PriceFeedInverse.sol

```
8 contract PriceFeedInverse is IPriceFeed {
```

same file

```
29     return int256(baseUnit.mulDivUp(baseUnit,
priceFeed.latestAnswer().toUint256()));
```

CVF-4: acknowledged Category: **behavior**

Decimals are cached at construction, so a change in the source feed's decimals would silently misprice wARS by orders of magnitude.

The constructor copies `priceFeed_.decimals()` into the immutable `decimals` and derives `baseUnit` from it. `latestAnswer` then computes $\lceil 10^{2d}/a \rceil$, with the cached d and whatever the source reports now. A Chainlink proxy can be re-pointed to a new aggregator, and the proxy's `decimals()` follows the current aggregator. If the source moved to $d' \neq d$ decimals, the result would be off by a factor of $10^{d'-d}$. The `Auditor` would

not notice: it checks `decimals()` only in `enableMarket` and `setPriceFeed`, and this contract keeps reporting the cached value.

At the current rate of about 1,608 ARS per USD, a switch from 8 to 18 decimals would make `latestAnswer` return 1, pricing wARS about 62,000 times too low. Accounts could then borrow almost the entire wARS market against negligible collateral. A switch from 8 to 6 decimals would overprice wARS about 100 times, making wARS borrowers liquidatable at once.

Chainlink rarely changes the decimals of an existing feed, and directly wrapped feeds carry a similar risk, so the likelihood is low. The inversion makes it worse in one respect, though: more source decimals now mean a lower price, which is the dangerous direction for a debt-only market.

Recommendation

In `latestAnswer`, check that the source still reports the cached decimals, and revert otherwise (fail closed):

```
uint8 sourceDecimals = priceFeed.decimals();
if (sourceDecimals != decimals) revert
  SourceDecimalsChanged(decimals, sourceDecimals);
```

If the extra call is considered too expensive on the liquidity-check path, at least monitor the source proxy off-chain for aggregator changes and compare the new aggregator's decimals before it is confirmed.

Client Comment

Acknowledged. PriceFeedInverse follows the same design as the protocol's other price feeds, which read the source's decimals once at construction. Chainlink changing an existing feed's decimals would be rare. Checking the source on every read would add an external call to every liquidity check. Changes to the source proxy's aggregator will be monitored off-chain, and the feed can be replaced through setPriceFeed if the decimals ever change.

/PriceFeedInverse.sol

```
21    decimals = priceFeed_.decimals();
22    baseUnit = 10 ** decimals;
```

same file

```
29    return int256(baseUnit.mulDivUp(baseUnit,
priceFeed.latestAnswer().toUint256()));
```

CVF-5: acknowledged Category: **behavior**

The inverted price keeps only as many significant digits as the source answer leaves, so its rounding error grows as the peso depreciates.

`latestAnswer` returns $\lceil 10^{2d}/a \rceil$ in the source's $d = 8$ decimals. `Auditor.enableMarket` and `Auditor.setPriceFeed` require exactly `priceDecimals = 8`, so the feed cannot report more digits. Rounding up adds less than one unit, so the relative error is below $1/\lceil 10^{2d}/a \rceil$, and that bound grows linearly with the USD/ARS rate:

- At the current 1,608.43 ARS per USD the result is 62,173 units against an exact 62,172.50, so the error is below 0.0016%.
- The error is below 0.01% at 10,000 ARS per USD, below 0.1% at 100,000, below 1% at 1,000,000, and below 10% at 10,000,000.
- From 10^8 ARS per USD up, every answer is 1, so the price no longer moves with the rate.

The bias is always upward. For a debt-only market, that overvalues wARS debt, which is conservative in `accountLiquidity` and `checkLiquidation`. In `calculateSeize`, however, a wARS price that is too high makes liquidators seize correspondingly more collateral from wARS borrowers. The error is negligible today, but the peso has lost more than three orders of magnitude against the dollar since 2001, so the bound deserves to be stated and watched.

Recommendation

Document the error bound in the contract, and monitor the rate. Revisit the design before the peso approaches about 100,000 per US dollar, where the error reaches 0.1%. Options at that point are a feed and market configuration that price a larger unit of wARS, or a higher `priceDecimals` in a future `Auditor` deployment.

Client Comment

Acknowledged. The precision is limited by the Auditor's 8 price decimals, which apply to every feed. The rate will be monitored, and the design will be revisited well before the error becomes significant.

/PriceFeedInverse.sol

```
29     return int256(baseUnit.mulDivUp(baseUnit,
priceFeed.latestAnswer().toUint256()));
```

CVF-6: acknowledged Category: **documentation**

The rounding-up rationale holds only while wARS stays non-collateral, which neither the contract nor its documentation requires.

The `@dev` comment justifies rounding up as "the conservative direction for debt-only markets". That is true for how the `Auditor` uses a non-collateral market's price: `accountLiquidity` and `checkLiquidation` apply it only to debt, and `handleBadDebt` never reads it. Nothing ties this feed to such a market, though. `Auditor.setNonCollateral(market, false)` can make wARS collateral at any time, and

the same contract could be reused for another market. The deployment script makes the mistake easy: `deploy/Markets.ts` defaults `nonCollateral` to `false` (line 188 at the reviewed commit), so a market entry that selects the inverse feed but omits the flag lists the market as collateral. In that case the upward bias would overvalue collateral in `accountLiquidity`, `checkLiquidation` and `handleBadDebt`, and would under-seize wARS in `calculateSeize`. The effect is below 0.0016% at today's rate, but it grows as the peso depreciates.

Consumers outside the protocol may not distinguish non-collateral markets either. The ExaPlugin's `ProposalManager.checkLiquidity` counts deposits in every entered market as collateral, wARS included, at the rounded-up price. That check runs off-chain, when the Exa server authorises card payments, so it matters as soon as Exa accounts can hold wARS.

The NatSpec also calls the source "the inverted feed" (in the comments on `decimals` and `baseUnit`), while the contract itself is the inverse feed. "Base unit of the inverted feed's answer" therefore reads as though it described this contract's output. The two happen to share a base unit, but the wording invites confusion.

Recommendation

State in the contract-level NatSpec that the feed is intended only for markets listed as non-collateral, and that its rounding is not conservative for collateral. Make the deployment scripts refuse the inverse feed for a market that is not marked non-collateral, and treat any later `setNonCollateral(false)` for such a market as requiring a review of the feed. Refer to the wrapped feed consistently as the "source feed", for example "Number of decimals of the answer, copied from the source feed".

Client Comment

Partially fixed in 7202c587c21ba7e4408e8570d3fed05eb856a73a. The NatSpec now refers to the wrapped feed as the "source feed". The rest is acknowledged. PriceFeedInverse isn't restricted to non-collateral markets, and its NatSpec already states that rounding up is the conservative direction for debt-only markets, which matches the intended use for wARS.

/PriceFeedInverse.sol

```
14  /// @notice Number of decimals that the answer of this price feed
has, same as the inverted feed's.
```

same file

```
16  /// @notice Base unit of the inverted feed's answer.
```

same file

```
26  /// @dev Rounds up, the conservative direction for debt-only
markets, so the inverse can't truncate to zero.
```

CVF-7: acknowledged Category: **documentation**

The adapter does not document that anyone can trigger `receiveFlashLoan` on any contract through it.

`flashLoan` is permissionless and accepts an arbitrary `recipient` and `data`. In a recipient's `receiveFlashLoan`, `msg.sender == adapter` therefore proves only that the call came through the adapter, not who started the loan. The current callers are safe because each also checks a hash that it stores just before borrowing, and deletes it before using the loan: `callHash` in `DebtRoller` and `flashLoaning` in the ExaPlugin extension. The NatSpec of the contract and of `flashLoan` says nothing about this requirement. A future integrator that authenticated the callback by `msg.sender` alone could have it driven by anyone, with attacker-chosen `data`.

Recommendation

State in the contract-level and `flashLoan` NatSpec that the function is permissionless, and that recipients must authenticate the initiator themselves, for example by storing a hash of the request before the call and clearing it in the callback. Also state that the recipient must transfer `amounts[0] + fees[0]` back to the adapter before returning.

Client Comment

Acknowledged. The adapter keeps the existing IFlashLoaner interface and behaves the same as the previous adapter. Both current integrations, DebtRoller and the ExaPlugin, already check who started the loan by verifying a stored request hash in receiveFlashLoan, and both transfer amounts[0] + fees[0] back to the adapter.

/periphery/FlashLoanAdapter.sol

```
9  /// @title FlashLoanAdapter
10 /// @notice Adapter to use Morpho flash loans with the `IFlashLoaner`
   interface.
11 ///   Morpho flash loans are free, so reported fees are always zero.
```

same file

```
22   /// @notice Performs a flash loan using Morpho.
23   /// @param recipient The address to receive the flash loan.
24   /// @param tokens The tokens to borrow.
25   /// @param amounts The amounts to borrow.
26   /// @param data Additional data to pass to the recipient.
```

CVF-8: acknowledged Category: **procedural**

No event records which recipient a loan was forwarded to.

Morpho emits `FlashLoan(caller, token, assets)`, in which `caller` is always the adapter, and the adapter itself emits nothing. The logs therefore do not show who requested a loan or which recipient received it. Telling ExaPlugin repays from `DebtRoller` rolls, or spotting third-party use of the adapter, requires transaction traces.

Recommendation

Emit an event in `flashLoan` that records `msg.sender`, `recipient`, the token and the amount, for example `event FlashLoan(address indexed caller, address indexed recipient, IERC20 indexed token, uint256 amount);` (with the recipient typed as `IFlashLoanRecipient` if the typing finding is adopted).

Client Comment

Acknowledged. The adapter behaves the same as the previous adapter and doesn't emit its own event. Morpho already emits FlashLoan with the token and amount. Every integration that uses the adapter repays or borrows through the market in the same transaction, and the market emits an event identifying the account.

/periphery/FlashLoanAdapter.sol

```
30     morpho.flashLoan(address(tokens[0]), amounts[0],
abi.encode(recipient, tokens, amounts, data));
```

CVF-9: fixed Category: **efficiency**

The `flashLoan` parameters could be `calldata`, and the linter suppression above them is now obsolete.

The `// solhint-disable-next-line gas-calldata-parameters` comment was needed in the Balancer V3 adapter, which rewrote `tokens[0]` and `amounts[0]` in memory on its wrapped-token path. The new `flashLoan` only reads its arguments and re-encodes them, so `tokens`, `amounts` and `data` can be `calldata`. That avoids copying both arrays and the data bytes into memory before `abi.encode` copies them again. An external function may change a parameter's data location when it overrides an interface function, so the `memory` declaration in `IFlashLoaner` does not prevent this.

Recommendation

Declare the parameters as `calldata` and remove the suppression comment:

```
function flashLoan(
  address recipient,
  IERC20[] calldata tokens,
  uint256[] calldata amounts,
  bytes calldata data
) external {
```

Client Comment

Fixed in 0eada6b32a7d91625b4f131475b9b96f15fd44ab

/periphery/FlashLoanAdapter.sol

```
27   // solhint-disable-next-line gas-calldata-parameters
```

```
28   function flashLoan(address recipient, IERC20[] memory tokens,
uint256[] memory amounts, bytes memory data) external {
```

CVF-10: fixed Category: **efficiency**

`latestAnswer` re-squares `baseUnit` on every call, and a source with unusable decimals is detected only when the price is first read

`latestAnswer` returns $\lceil \text{baseUnit}^2 / \text{answer} \rceil$, computed as:

```
baseUnit.mulDivUp(baseUnit, answer)
```

`baseUnit` is an `immutable` set once in the constructor to 10^d (`baseUnit = 10 ** decimals`), so the exponentiation 10^d is not repeated per call. What recurs on every call is the squaring: `mulDivUp` multiplies `baseUnit` by itself and checks that product for overflow, even though $\text{baseUnit}^2 = 10^{2d}$ is a fixed deployment constant that could be precomputed once. The `Auditor` reads every entered market's price in each liquidity check, so this redundant multiply-and-overflow-check recurs on hot paths.

The per-call overflow check has a second effect. A source with 39 to 77 decimals passes the constructor, because `baseUnit = 10 ** decimals` still fits in 256 bits (up to 10^{77}), but then every `latestAnswer` call reverts with `MulDivFailed()`, because the product $\text{baseUnit} \cdot \text{baseUnit} = 10^{2d}$ overflows 256 bits once $2d > 77$, i.e. $d \geq 39$. The misconfiguration therefore surfaces only after deployment, on the first price read, rather than at construction.

Finally, the public `baseUnit` getter is not used by the `Auditor`, `Previewer`, `DebtPreviewer`, or the ExaPlugin.

Recommendation

Precompute the squared numerator once in the constructor and divide with rounding up, so the per-call squaring and its overflow check disappear and an unusable `decimals` fails at deployment:

```
uint256 internal immutable numerator; // 10 ** (2 * decimals)

constructor(IPriceFeed priceFeed_) {
  priceFeed = priceFeed_;
  decimals = priceFeed_.decimals();
  numerator = 10 ** (2 * uint256(decimals));
}

function latestAnswer() external view returns (int256) {
  return
  int256(numerator.divUp(priceFeed.latestAnswer().toUint256()));
}
```

The `10 ** (2 * decimals)` then overflows at deployment for any $d > 38$, surfacing the bad source immediately, and the result still fits in `int256`. Unless an off-chain consumer needs `baseUnit`, drop that getter or make it internal.

Client Comment

Fixed in 902392f009066b1dd8fc194aa9b626fb21374b82

/PriceFeedInverse.sol

```
17   uint256 public immutable baseUnit;
```

same file

```
22     baseUnit = 10 ** decimals;
```

same file

```
29     return int256(baseUnit.mulDivUp(baseUnit,
priceFeed.latestAnswer().toUint256()));
```

CVF-11: acknowledged Category: **procedural**

The adapter neither declares nor imports Morpho's callback interface, so the compiler does not check the `onMorphoFlashLoan` signature.

Morpho calls the borrower back as follows:

```
IMorphoFlashLoanCallback(msg.sender).onMorphoFlashLoan(assets, data);
```

`FlashLoanAdapter` inherits only `IFlashLoaner`, and `IMorpho` is a hand-written one-function excerpt of Morpho's interface. A typo in the callback's name or parameter types would compile without complaint and make every loan revert at run time. The signature is correct today, but only the tests guarantee it.

Recommendation

Declare `IMorphoFlashLoanCallback`, or import it together with `IMorpho` from the Morpho Blue interfaces, and inherit it:

```
contract FlashLoanAdapter is IFlashLoaner,
  IMorphoFlashLoanCallback {
```

Client Comment

Acknowledged. This is a code style decision. The adapter follows the same pattern as the previous adapter, which didn't inherit an interface for its vault callback either. The callback signature is covered by the tests, which exercise every loan through Morpho.

/periphery/FlashLoanAdapter.sol

```
12 contract FlashLoanAdapter is IFlashLoaner {
```

same file

```
35   function onMorphoFlashLoan(uint256, bytes calldata payload)
external {
```

same file

```
56 interface IMorpho {
57   function flashLoan(address token, uint256 assets, bytes calldata
data) external;
58 }
```

CVF-12: acknowledged Category: **typing**

`recipient` is typed as `address`, although it is always called as an `IFlashLoanRecipient`.

`flashLoan` takes `address recipient`, the payload decodes it as `address`, and `onMorphoFlashLoan` converts it with `IFlashLoanRecipient(recipient)` before calling it. Balancer V2's own `IVault.flashLoan`, whose interface `IFlashLoaner` mirrors, types this argument as `IFlashLoanRecipient`. The more precise type would document the expectation and let the compiler check it.

Recommendation

Type the parameter as `IFlashLoanRecipient` in `IFlashLoaner.flashLoan` and in the adapter, decode it with that type, and convert to `address` only where a raw address is needed, such as in `safeTransfer`. Interface types are ABI-encoded as `address`, so the selector does not change and `DebtRoller` and the ExaPlugin stay compatible.

Client Comment

Acknowledged. The adapter keeps the previous adapter's interface, where the recipient is an address, matching IFlashLoaner and the ExaPlugin's copy of the interface.

/periphery/FlashLoanAdapter.sol

```
28   function flashLoan(address recipient, IERC20[] memory tokens,
uint256[] memory amounts, bytes memory data) external {
```

same file

```
37     (address recipient, IERC20[] memory tokens, uint256[] memory
amounts, bytes memory data) = abi.decode(
38       payload,
39       (address, IERC20[], uint256[], bytes)
40     );
```

same file

```
44      IFlashLoanRecipient(recipient).receiveFlashLoan(tokens, amounts,
new uint256[](1), data);
```

CVF-13: acknowledged Category: **documentation**

NatSpec is missing for the constructors, return values, the Morpho interface, the errors and the unnamed callback parameter.

In `PriceFeedInverse`, the constructor and its `priceFeed_` parameter are undocumented, and `latestAnswer` has no `@return` that states the unit (US dollars per peso, with `decimals` decimals) and the rounding.

In `FlashLoanAdapter`, the constructor and `morpho_` are undocumented, and `available` has no `@return`. `IMorpho` and the two errors have no NatSpec at all. `onMorphoFlashLoan` leaves its first parameter, Morpho's `assets`, unnamed and undocumented; using it in place of the payload's `amounts[0]` is covered by the finding on the redundant flash-loan payload.

Recommendation

Add the missing `@param`, `@return` and `@notice` tags, including a name and description for the first parameter of `onMorphoFlashLoan`.

Client Comment

Acknowledged. The NatSpec follows the conventions used across the repo, which don't document constructors, errors or minimal external interfaces. In both in-scope contracts, the view functions latestAnswer and available describe their return value in the @notice, as the protocol's other price feeds do.

/PriceFeedInverse.sol

```
19    constructor(IPriceFeed priceFeed_) {
```

same file

```
25    /// @notice Returns the inverse of the price feed's latest answer.
26    /// @dev Rounds up, the conservative direction for debt-only
markets, so the inverse can't truncate to zero.
27    /// Reverts on a zero or negative answer.
```

/periphery/FlashLoanAdapter.sol

```
18    constructor(IMorpho morpho_) {
```

same file

```
33    /// @notice Receives a flash loan from Morpho, forwards it to the
recipient and approves the repayment.
34    /// @param payload The payload containing the recipient, tokens,
amounts, and data.
```

```
35   function onMorphoFlashLoan(uint256, bytes calldata payload)
external {
```

same file

```
49   /// @notice Returns the amount of `token` available to be flash
loaned.
50   /// @param token The token to check.
51   function available(IERC20 token) external view returns (uint256) {
```

same file

```
56 interface IMorpho {
```

same file

```
60 error InvalidLength();
```

same file

```
61 error UnauthorizedMorpho();
```

CVF-14: fixed Category: **readability**

In `PriceFeedInverse.sol`, the local import is not separated from the third-party imports.

The local `./utils/IPriceFeed.sol` import directly follows the two Solady imports. `FlashLoanAdapter.sol` separates the same two groups with an empty line, so the two in-scope files follow different conventions. Separating third-party and local imports with an empty line makes a file's dependencies easier to see.

Recommendation

Insert an empty line between the Solady imports and the local import.

Client Comment

Fixed in 778e7371ba670061d77acacd5e3827972c93d8af

/PriceFeedInverse.sol

```
4 import { FixedPointMathLib } from
"solady/src/utils/FixedPointMathLib.sol";
5 import { SafeCastLib } from "solady/src/utils/SafeCastLib.sol";
6 import { IPriceFeed } from "./utils/IPriceFeed.sol";
```

CVF-15: acknowledged Category: **typing**

Parallel `tokens` and `amounts` arrays require a manual length check

The `flashLoan` function accepts `IERC20[] memory tokens` and `uint256[] memory amounts` as separate parallel arrays, where element `i` of one is meant to correspond to element `i` of the other. Because the lengths are carried independently, the function must guard the correspondence by hand with `if (tokens.length != 1 || amounts.length != 1) revert InvalidLength();`, and the same parallel pair is rematerialised when the payload is decoded in `onMorphoFlashLoan` and then forwarded as three parallel arrays (`tokens`, `amounts`, and the freshly allocated fee array) into `receiveFlashLoan`.

Parallel arrays are suboptimal because:

- The length of each array is passed independently even though the two are always expected to line up, so an inconsistent-length call is representable and must be rejected at runtime.
- The contract has to spend code and gas validating that the arrays agree (`InvalidLength()`), a check that a single array of pairs would make structurally impossible.

A struct array such as `struct FlashLoanRequest { IERC20 token; uint256 amount; }` passed as `FlashLoanRequest[] memory requests` would make each token bound to its amount by construction and remove the need for the length guard. Note that the `tokens`/`amounts` shape is imposed by the external `IFlashLoaner`/`IFlashLoanRecipient` interface (imported from `DebtRoller.sol`), so changing it here also requires changing that interface; if Balancer-style interface compatibility must be preserved, this is a design observation rather than an immediately actionable change.

Recommendation

Where the interface is under your control, merge the parallel `tokens` and `amounts` arrays into a single array of a struct that binds each token to its amount, e.g.:

```
struct FlashLoanRequest {
  IERC20 token;
  uint256 amount;
}

function flashLoan(address recipient, FlashLoanRequest[] memory
  requests, bytes memory data) external {
  ...
}
```

This makes mismatched lengths unrepresentable and lets the `InvalidLength()` check be dropped. If the `tokens`/`amounts` shape must stay to remain compatible with the external `IFlashLoaner`/`IFlashLoanRecipient` interface declared in `DebtRoller.sol`, consider changing it at the interface level, or document that the parallel-array form is an intentional compatibility constraint.

Client Comment

Acknowledged. The parallel tokens and amounts arrays are part of the IFlashLoaner interface kept from the previous adapter. Changing them would change the selector

and break existing integrations.

/periphery/FlashLoanAdapter.sol

```
28   function flashLoan(address recipient, IERC20[] memory tokens,
uint256[] memory amounts, bytes memory data) external {
```

same file

```
29     if (tokens.length != 1 || amounts.length != 1) revert
InvalidLength();
```

same file

```
37     (address recipient, IERC20[] memory tokens, uint256[] memory
amounts, bytes memory data) = abi.decode(
38       payload,
39       (address, IERC20[], uint256[], bytes)
40     );
```

same file

```
44     IFlashLoanRecipient(recipient).receiveFlashLoan(tokens, amounts,
new uint256[](1), data);
```

CVF-16: fixed Category: **efficiency**

Flash loan payload encodes redundant `tokens` and `amounts` data

When initiating the flash loan, `flashLoan` encodes the full `tokens` and `amounts` arrays into the payload handed to Morpho:

```
morpho.flashLoan(address(tokens[0]), amounts[0],
  abi.encode(recipient, tokens, amounts, data));
```

Both of these are redundant:

- `tokens` is encoded as a dynamic array even though the function has already guaranteed it holds exactly one element — the preceding guard `if (tokens.length != 1 || amounts.length != 1) revert InvalidLength();` reverts otherwise. Encoding the whole array stores an extra length word plus array-offset overhead, when only `tokens[0]` carries information.
- `amounts` is likewise single-element, and its one value is data Morpho already supplies: Morpho invokes the callback as `onMorphoFlashLoan(uint256 assets, bytes calldata data)`, passing the loaned amount as the first argument. The adapter currently ignores that argument (it is unnamed in the `onMorphoFlashLoan` signature) and reads `amounts[0]` out of the decoded payload instead, so the encoded `amounts` duplicates a value that is available for free in the callback.

The effect is a larger ABI-encoded payload on every flash loan — more calldata, memory, and gas — carrying values that are either fixed by an invariant or already delivered by Morpho.

Note that the decoded `tokens` and `amounts` are forwarded to `receiveFlashLoan(tokens, amounts, new uint256[](1), data)`, which expects `IERC20[]` and `uint256[]`. Applying this optimization therefore requires rebuilding one-element arrays inside the callback from the single token and the callback's amount argument before calling `receiveFlashLoan`.

Recommendation

Encode `tokens[0]` (a single `address`) rather than the whole `tokens` array, and omit `amounts` from the payload entirely. In `onMorphoFlashLoan`, name and use the first `uint256` parameter as the loan amount instead of reading it back from the payload, and reconstruct the one-element `tokens` and `amounts` arrays locally for the `receiveFlashLoan` call. For example:

```
// in flashLoan:
morpho.flashLoan(address(tokens[0]), amounts[0],
  abi.encode(recipient, tokens[0], data));

// in onMorphoFlashLoan:
function onMorphoFlashLoan(uint256 amount, bytes calldata payload)
  external {
  if (msg.sender != address(morpho)) revert UnauthorizedMorpho();
  (address recipient, IERC20 token, bytes memory data) =
    abi.decode(payload, (address, IERC20, bytes));

  IERC20[] memory tokens = new IERC20[](1);
  tokens[0] = token;
  uint256[] memory amounts = new uint256[](1);
  amounts[0] = amount;

  address tokenAddr = address(token);
  tokenAddr.safeTransfer(recipient, amount);
  IFlashLoanRecipient(recipient).receiveFlashLoan(tokens, amounts,
  new uint256[](1), data);
  tokenAddr.safeApprove(address(morpho), amount);
}
```

Client Comment

Fixed in 81a5d64a7e0adfb2c8df69043ce3826a458643f2

/periphery/FlashLoanAdapter.sol

```
30     morpho.flashLoan(address(tokens[0]), amounts[0],
abi.encode(recipient, tokens, amounts, data));
```

6. Conclusion

This audit of branch `morpho-flashloan-adapter` of exactly/protocol at commit `371e41a`, covering the new `PriceFeedInverse.sol` and the rewritten `FlashLoanAdapter.sol` (53 nSLOC), produced sixteen findings, all of minor severity; no critical, major or moderate issues were found. The client's fixes were reviewed at commit `9bf6024`. Four findings are fixed (`CVF-9`, `CVF-10`, `CVF-14` and `CVF-16`), and the other twelve are acknowledged, two of them, `CVF-3` and `CVF-6`, after partial fixes. Every fix was checked against the source, and the code the fix round adds, chiefly the round data forwarding in `PriceFeedInverse`, introduces no new issue. The flash-loan callback is reachable only through the adapter's own Morpho loans, the callback authentication of `DebtRoller` and the ExaPlugin remains intact, and no allowance survives a loan.

The main acknowledged risks concern how the adapter is used. It still does not check that the recipient repaid, so any token balance left in the adapter can be taken by anyone through a loan that this balance repays (`CVF-1`); the client notes that the adapter is not meant to hold tokens and that both callers return exactly the borrowed amount. A loan larger than Morpho's balance still reverts with Morpho's generic message, and Morpho on Base holds no wARS (`CVF-2`). The client will have off-chain components cap flash-loan amounts with `available()`, and wARS debt will be repaid directly instead of through flash loans. Because `flashLoan` stays permissionless and emits no event of its own, every integrator must authenticate the loan's initiator itself, as `DebtRoller` and the ExaPlugin do (`CVF-7`, `CVF-8`).

The protocol still reads the inverse price through `latestAnswer` without bounding its age, a trade-off the client accepts for all its feeds (`CVF-3`). The partial fix forwards the source's round data through `latestRoundData`, `getRoundData` and `phaseId`, so monitoring can now see when the price was last updated, but a stalled source would still go unnoticed on-chain. Decimals remain cached at construction (`CVF-4`); the client will monitor the source proxy's aggregator off-chain and replace the feed through `setPriceFeed` if its decimals change. The rounding error of the inverse, below 0.0016% today, grows as the peso depreciates (`CVF-5`); the client will monitor the rate and revisit the design well before the error becomes significant. For `CVF-6`, the NatSpec now names the source feed consistently, while the feed itself is still not restricted to the debt-only markets for which rounding up is conservative.

Of the code-quality points, `flashLoan` now takes `calldata` parameters (`CVF-9`), the squared base unit is precomputed so that an unusable source fails at deployment (`CVF-10`), the imports are grouped (`CVF-14`), and the Morpho payload no longer carries the one-element arrays (`CVF-16`). The missing Morpho callback interface (`CVF-11`), the `address` type of `recipient` (`CVF-12`), the missing NatSpec (`CVF-13`) and the parallel `tokens` and `amounts` arrays

(CVF-15) were acknowledged, mostly to keep the IFlashLoaner interface and the conventions of the previous adapter.

While reading the surrounding code, we also noticed three issues outside the audit scope. First, the new DebtRoller decodes six fields from auditor.markets(), while the Auditor deployed today returns five; on Base, the Auditor upgrade that adds non-collateral markets must therefore execute before the DebtRoller upgrade, or every roll reverts until it does. Second, ProposalManager.checkLiquidity in the ExaPlugin, which the Exa server uses to authorise card payments, does not read the new nonCollateral flag and counts non-collateral deposits, such as wARS, as cover. Third, deploy/mocks/Assets.ts has no guard against live networks: it skips a mock price feed only when a deployment of that name already exists, so if the inverse-feed deployment file were missing on Base, it would deploy a mock feed that prices wARS at one US dollar. The timelock on market listing limits the damage.

The assessment of the price feed assumes that the wARS market is listed as non-collateral and stays so, which the client confirms is the intended use, and that the deployed feed is wired to Chainlink's USD/ARS proxy on Base, whose aggregator the client will monitor, rather than to an underlying aggregator, which would not serve phaseId. wARS, Ripio's peso token, follows a market dollar rate, as the feed does, rather than the official rate; it is pausable and upgradeable by its issuer, which is outside the protocol's control.

Overall, the reviewed code is small and straightforward, and the fix round improved it without introducing new issues. The residual risks are the acknowledged ones above, chiefly CVF-1 to CVF-5, and they rest on the client's stated measures: direct repayment of wARS debt, off-chain use of available(), and monitoring of the USD/ARS feed and its aggregator.



ABDK
Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

EMail

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

@ABDKConsulting

LinkedIn

company/abdk-consulting